

RELATÓRIO TÉCNICO

Sistema de Controle de Acesso com Arduino

Disciplina:

IOT

Alunos:

Danilo Cardoso Pradella (RA: 22.126.096-1)

Caue Ohta (RA: 22.126.093-8)

<https://github.com/mixsz/IOT>

1. Introdução

Este relatório descreve o desenvolvimento de um **sistema eletrônico de controle de acesso** para um cofre, implementado em plataforma Arduino Uno e simulado no ambiente Tinkercad. O projeto integra hardware e firmware para gerenciar a entrada de uma senha numérica, validar o acesso e monitorar condições de segurança do ambiente, atendendo ao objetivo proposto pela atividade.

O sistema opera por meio de uma máquina de estados finitos com quatro estados distintos (TRANCADO, SENHA, ABERTO e ALERTA), acionados por entradas de hardware como potenciômetro, botões e sensor de luz, e realimentados ao usuário por display LCD e buzzer.

2. Objetivo

Projetar e implementar um sistema embarcado capaz de:

- Receber uma senha de 4 dígitos via potenciômetro e botão de confirmação;
- Validar a senha digitada e acionar um servo motor como mecanismo de trava;
- Emitir alertas sonoros e visuais em caso de muitas tentativas de erro ou tentativa de arrombamento;
- Detectar abertura forçada por meio de sensor de luminosidade (LDR);
- Permitir reset imediato do sistema via interrupção externa de hardware.

3. Hardware

3.1 Lista de Componentes

Componente	Qtd.	Função
Arduino Uno	1	Microcontrolador principal, executa o firmware e gerencia todos os periféricos
Display LCD 16x2	1	Exibe o estado do sistema (FECHADO, ABERTO, ALERTA) e a senha em digitação
Servo Motor	1	Atua mecanicamente como trava, onde: 0° = fechado 90° = aberto
Potenciômetro 50kΩ	2	Ajuste de contraste do LCD e entrada analógica para selecionar dígito da senha
Botão de dígito	1	Confirma o dígito selecionado pelo potenciômetro
Botão de reset	1	Interrupção externa, reseta o sistema para o estado inicial, que seria FECHADO
Buzzer	1	Feedback sonoro: sucesso, erro e alarme de segurança
LDR	1	Sensor de luz, detecta arrombamento quando o cofre é aberto a força
Resistor 300Ω	2	Limitação de corrente (buzzer e LDR)

Protoboard	1	Plataforma de montagem e conexão dos componentes
------------	---	--

3.2 Diagrama de conexão

O diagrama a seguir apresenta as conexões elétricas do sistema, gerado no Tinkercad:

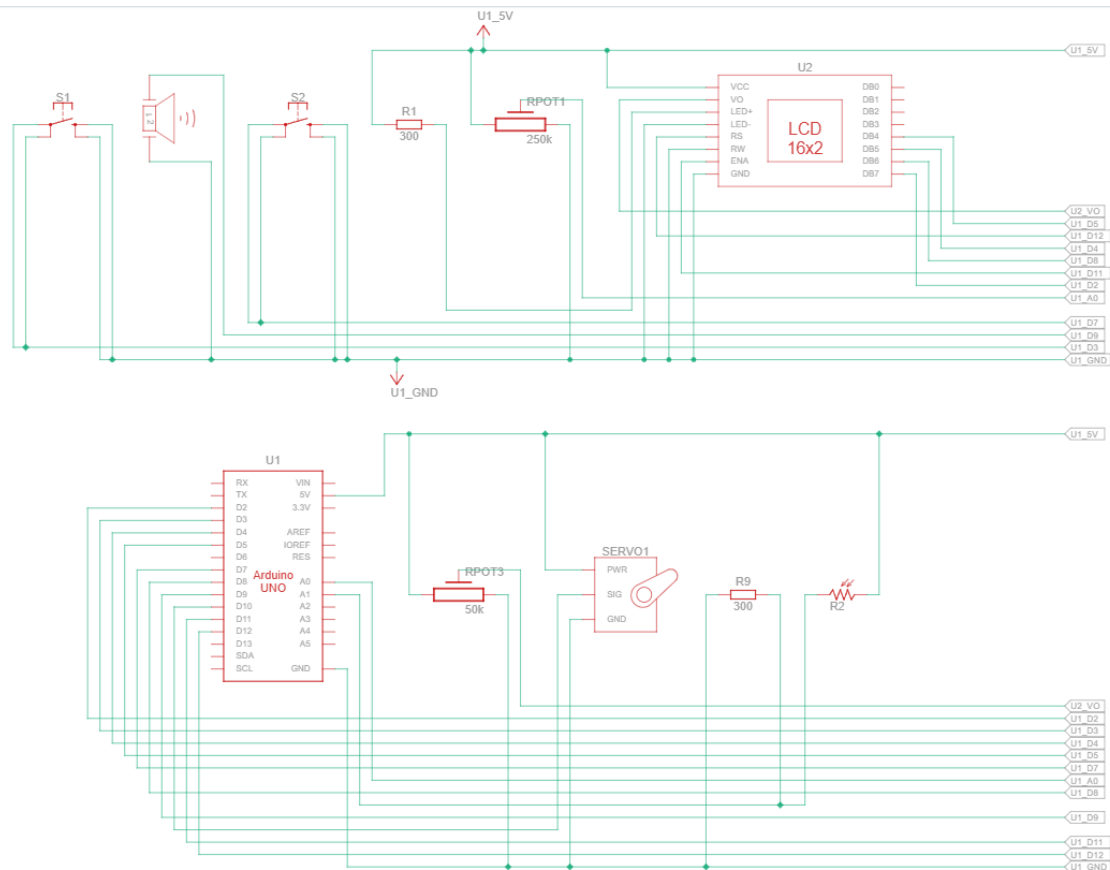


Figura 1: Diagrama de conexão do circuito

3.3 Circuito Montado (Protoboard)

A figura abaixo ilustra o circuito montado na protoboard com todos os componentes interligados ao Arduino Uno:

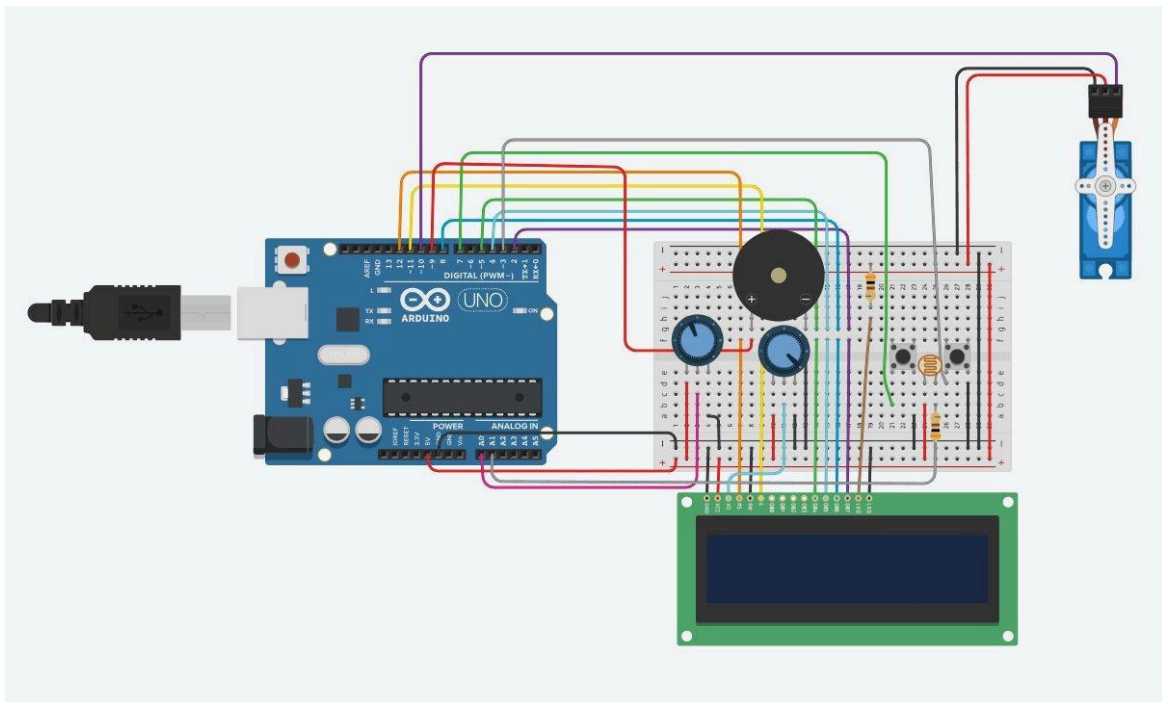


Figura 2: Circuito feito no tinkercad

3.4 Mapeamento de Pinos

Pino Arduino	Periférico	Descrição
D2	LCD DB7	Barramento de dados do LCD
D4	LCD DB5	Barramento de dados do LCD
D5	LCD DB4	Barramento de dados do LCD
D7	LCD EN	Enable do LCD
D8	LCD DB3	Barramento de dados do LCD
D9	Buzzer	Saída PWM para geração de tons
D10	Servo (PWM)	Controle de posição do servo motor
D11	LCD RS	Register Select do LCD
D12	LCD D6	Barramento de dados do LCD
D3	Botão Reset	Interrupção externa INT1 (FALLING)
D7	Botão Dígito	Entrada digital com pull-up interno
A0	Potenciômetro	Leitura analógica para seleção de dígito
A1	LDR	Leitura analógica para detecção de luz

4. Firmware

4.1 Linguagem e Bibliotecas

O firmware foi desenvolvido em linguagem C++ para Arduino, utilizando as seguintes bibliotecas nativas:

- LiquidCrystal.h: controle do display LCD 16x2 via interface paralela de 4 bits;
- Servo.h: geração do sinal PWM para controle de posição do servo motor.

4.2 Máquina de Estados

O comportamento do sistema é governado por uma máquina de estados finitos (FSM) definida pelo tipo enumerado Estado, com quatro estados:

Estado	Condição de Entrada	Comportamento
TRANCADO	Inicialização / senha incorreta / reset	Servo em 0°, LCD exibe 'FECHADO' e senha zerada
SENHA	Usuário move o potenciômetro	LCD mostra dígito atual e botão confirma o dígito
ABERTO	Senha de 4 dígitos correta	Servo em 90°, LCD exibe 'ABERTO' e só sai via reset
ALERTA	3 tentativas erradas ou LDR detecta luz	Buzzer contínuo, LCD exibe mensagem de alarme e só sai via reset

4.3 Lógica de Entrada da Senha

A senha é composta por **4 dígitos no intervalo de 0 a 4**. Cada dígito é selecionado girando o potenciômetro conectado à entrada analógica A0, cujo valor (0 a 1023) é mapeado em faixas para os caracteres '0' a '4'. O botão de dígito (D7, com pull-up interno) confirma a seleção com detecção de borda de descida (HIGH → LOW), evitando leituras repetidas.

A função ler_digito() implementa o mapeamento analógico:

```
v < 150 → '0' | v < 350 → '1' | v < 550 → '2' | v < 750 → '3' |  
demais → '4'
```

Uma tolerância de 20 pontos ADC é utilizada para filtrar oscilações e confirmar que o usuário efetivamente moveu o potenciômetro antes de aceitar um dígito.

4.4 Validação e Controle de Tentativas

Após a inserção dos 4 dígitos, o firmware compara caractere a caractere com a senha correta que é armazenada no código. Em caso de sucesso, o sistema transita para ABERTO. Em caso de falha:

- O contador de tentativas é incrementado e o número de chances restantes é exibido no LCD;
- Após 3 tentativas incorretas, o sistema entra em ALERTA, com buzzer contínuo pulsante e bloqueio total até o reset.

4.5 Detecção de Arrombamento (LDR)

O sensor LDR está posicionado de modo que, com o cofre fechado, receba pouca luz. Se o valor analógico lido em A1 superar o limiar de **290** durante os estados TRANCADO ou SENHA, o firmware interpreta isso como um arrombamento, define a flag **arrombado = true** e transita imediatamente para ALERTA, exibindo a mensagem 'COFRE ARROMBADO!' e acionando o buzzer em frequência de perigo.

4.6 Reset por Interrupção

O botão de reset está conectado ao pino D3, configurado para disparar na borda de descida. A Rotina de Serviço de Interrupção (ISR) apenas seta a flag `reset_solicitado = true`, que é verificada com prioridade máxima no início de cada iteração do `loop()`, garantindo resposta imediata independente do estado atual.

4.7 Feedback Sonoro

O buzzer emite padrões distintos para cada evento:

- Sucesso (senha correta): sequência de 3 notas;
- Erro (senha incorreta): sequência descendente de 2 notas;
- Alerta (tentativas esgotadas): pulsos alternados em 880 Hz e 660 Hz;
- Perigo (arrombamento): pulsos em 880 Hz com intervalos de 100 ms.

5. Fluxo de Operação

O sistema opera conforme o fluxo abaixo a cada iteração do loop principal:

1. Verificar flag de reset → se ativada, executar `processar_reset()` e retornar.
2. Verificar LDR → se luz acima do limiar em estado fechado, acionar ALERTA de arrombamento.
3. Processar o estado atual:
 - TRANCADO / SENHA → ler potenciômetro, exibir dígito, processar botão de confirmação.
 - ABERTO → aguardar reset (sem transição automática).
 - ALERTA → emitir buzzer contínuo até reset.

6. Resultados

O sistema foi simulado com sucesso no Tinkercad, demonstrando o funcionamento correto de todos os estados. Os pontos observados foram:

- A máquina de estados mostrou-se robusta, com transições previsíveis e sem estados intermediários inesperados;
- A leitura por faixas do potenciômetro evitou ambiguidades na seleção de dígitos;
- A interrupção de hardware garantiu resposta imediata ao reset independentemente do estado, incluindo durante a execução de delays no processamento da senha;
- O LDR provou ser eficaz como sensor anti-arrombamento dentro das condições de simulação;
- Os padrões sonoros distintos do buzzer mostraram-se suficientes para diferenciar os eventos de forma intuitiva.

7. Conclusão

O projeto demonstrou com sucesso a viabilidade de implementar um sistema de controle de acesso funcional utilizando componentes eletrônicos acessíveis e a plataforma Arduino. A abordagem por máquina de estados finitos mostrou-se adequada para o problema, facilitando a leitura, manutenção e extensão do código.

Os objetivos estabelecidos foram integralmente atingidos: entrada de senha analógica, validação, controle de acesso físico por servo, alarme sonoro e visual, detecção de arrombamento por LDR e reset por interrupção de hardware. O projeto constitui uma base sólida para evoluções futuras, como adição de teclado matricial, conectividade sem fio ou registro de eventos.